

F7: Abstrakta klasser, interfaces och polymorfism

Malmö universitet, Institutionen för datavetenskap och
medieteknik

Innehåll

- Repetition
- Abstrakta klasser
- Interfaces
- Polymorfism
- Sammanfattning



Repetition

Repetition

- En klass kan ärva egenskaper av en annan klass
- Super- vs. sub-klasser
- *protected* gör variabler tillgängliga i hela paketet (tillskillnad från *private*)
- En subklass kan använda sin superklass metoder, eller skriva över med sina egna
- En superklass behöver inte visa att den kan ärvas från
- Sub-klasser *extends* super-klasser
- Generellt kallas detta för *generalisering*



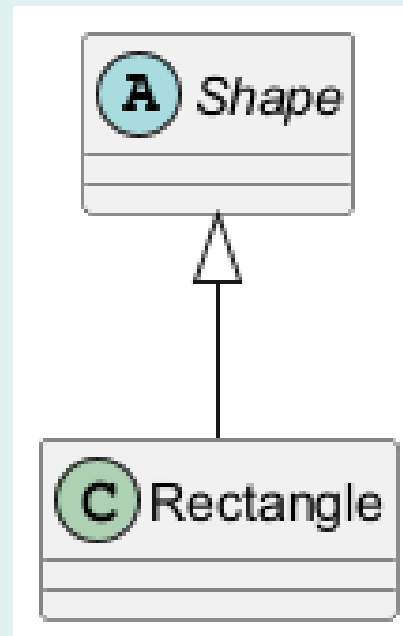
Abstrakta klasser

Abstrakta klasser

- En *abstrakt* klass är en klass som inte kan instansieras
- En *abstrakt* klass måste alltså ärvas
- En *abstrakt* klass kan attribut och operationer som vanligt som kan ärvas

Klassdiagram

- I ett klassdiagram markerar man att en klass (eller en metod) är abstrakt genom att *kursivera* namnet.



Abstrakta metoder

- En abstrakt metod är en metod utan metodkropp
- En klass som innehåller abstrakta metoder är också abstrakt
 - Det innebär att en klass som ärver abstrakta metoder utan att skriva över dem också är abstrakt

Varför?

- Föreställ dig att du har ett datorspel med en massa 3-dimensionella block som man kan slå sönder
- Det kan exempelvis vara block av typen *Dirt*, *Stone* eller *Clay*
- Dessa ärver alla attribut och operationer från ett ”ur-block”
- Detta ”ur-block” ska däremot aldrig ritas ut eller finnas i spelet.



Interfaces

Vad är ett interface?

- Ett interface är en ärvbar struktur med bara abstrakta metoder
- En klass kan ärva från flera interfaces

Interface vs. abstrakt klass

- En klass kan ärvas från en annan klass
- En klass kan ärvas från flera olika interfaces
- En abstrakt klass kan ha attribut
- Ett interface har bara konstanta attribut
- En abstrakt klass har abstrakta metoder
- Ett interface har abstrakta metoder
- En abstrakt klass kan inte instansieras
- Ett interface kan inte instansieras

Varför interfaces?

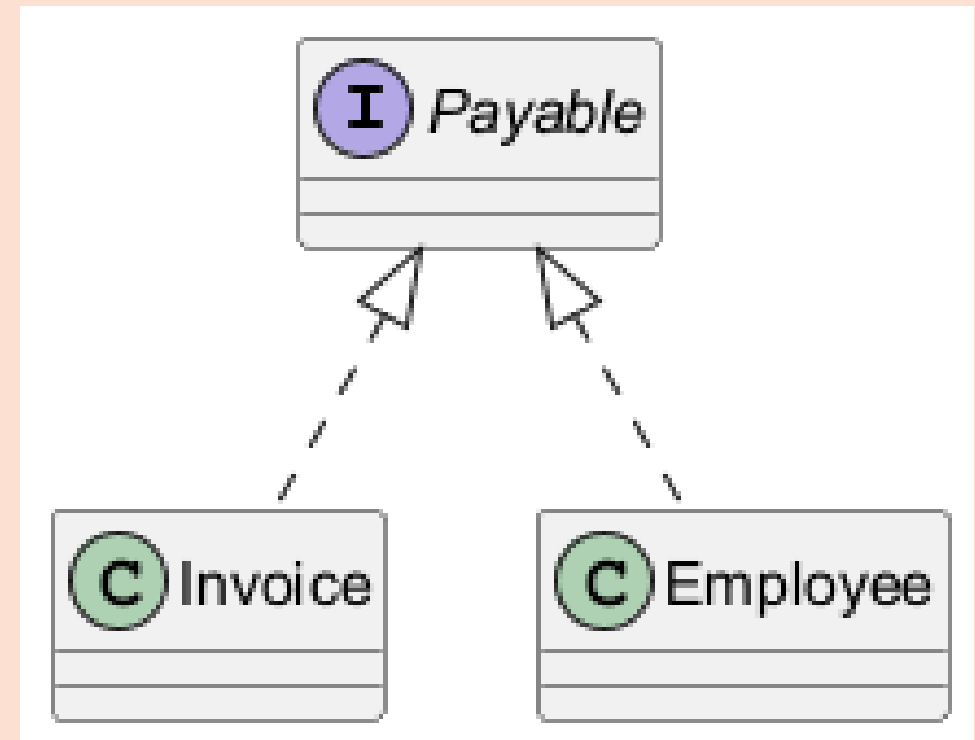
- En klass kan ärva flera interfaces
- Säger inget om hur något ska implementeras
- Ger stor frihet vid kodning

Lite exempel

- Ett spel där vissa objekt är *interactable*
- Eller en fjärrkontroll
 - Vi är överens om att en fjärrkontroll ska göra vissa saker
 - Vi är inte överens om *hur* den ska göra dessa saker
- En bil har flera interfaces
 - Vi kan accelerera, bromsa, svänga
 - Detta kan göras utan pedaler eller ratt.

Interfaces i klassdiagram

- Markeras likt en abstrakt klass med kursiv text
- Har likt generalisering tomma pilar
- Har streckade pilar



Interfaces i kod

```
public interface MyInterface {  
  
    public abstract int getValue();  
    public abstract void setVale(int value);  
    public abstract void doSomething();  
  
}  
public class MyClass implements MyInterface{  
  
}
```



Polymorfism

Polymorfism

- När vi instansierar en subklass istället för en superklass

```
Block myBlock1 = new Clay();  
Block myBlock2 = new Dirt();  
Block myBlock3 = new Stone();
```

Användbarhet

- Gör det möjligt att spara likartade objekt i samma array
- I ett program med ett GUI hade du kunnat ha alla ”fönsterobjekt” i samma array
- En butik hade kunnat ha en array *varukorg* med olika sorters varu-objekt
- Att överskugga metoder är en del av polymorfism



Sammanfattning

Abstrakta klasser

- Detta är klasser som har abstrakta metoder
 - en abstrakt metod har ingen metod-kropp
- Kan inte instansieras
- En klass måste vara abstrakt om den innehåller abstrakta metoder

Interfaces

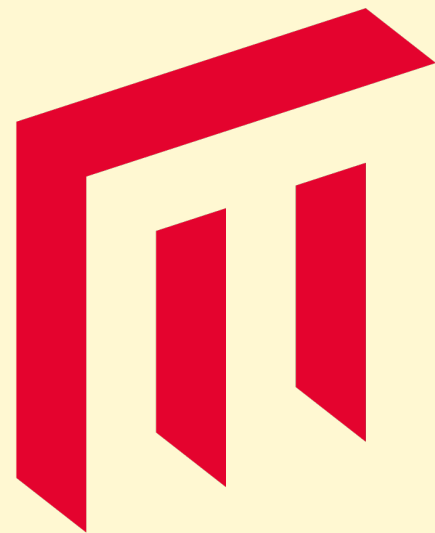
- Ett interface har enbart abstrakta metoder
 - Sanning med modifikation
- En klass kan implementera flera olika interfaces

Polymorfism

- När den exakta datatypen inte är bestämd vid deklarereringen av variabeln
- Möjliggör att olika subklasser kan instansieras som en superklass
- Används ofta med arrayer
- En subklass kan override:a en superklass metod

Rekommenderad läsning

- Deitel & Deitel, kapitel 10, s. 417–464
- J. R. R. Tolkien, *Sagan om ringen*



**MALMÖ
UNIVERSITET**

